SESSION 1 OF 3

Verilog HDL

Foundations & Language Basics



History, Introduction, Specification, Module, Data Types & Operators

History · Introduction · Specification · Module
Data Types · Operators

SESSION 1 AGENDA

Session 1 Overview

6 sections — language origins through core syntax and operators

1. History
2. Introduction
3. Specification
4. Module
5. Data Types
6. Operators

1 · HISTORY

History of Verilog

From an in-house company tool to an industry standard

Origins at Gateway Design Automation

- 1984 — Phil Moorby developed Verilog at Gateway Design Automation
- Designed to work with the company's own proprietary simulator
- 1989 — Cadence Design Systems acquired Gateway Design Automation
- 1990 — Cadence opened the language to the public domain to compete with VHDL

Why Go Public?

- VHDL (developed for the U.S. military) was already an open standard
- The industry wanted a language not tied to a single vendor
- Openness helped Verilog grow rapidly among engineers

1 · HISTORY

The Path to an IEEE Standard

From Verilog-95 to SystemVerilog

1995

IEEE 1364-1995

Verilog officially became an IEEE standard (Verilog-95)

2001

IEEE 1364-2001

Added generate blocks, multi-dimensional arrays, new operators

2005

IEEE 1364-2005

Minor refinements, improved compatibility

2005+

SystemVerilog

Extensions for verification & design (IEEE 1800)

1 · HISTORY

Verilog vs VHDL

Two dominant HDLs in industry — a comparison of design philosophy

Aspect	Verilog	VHDL
Origin	Gateway Design Automation (1984)	U.S. Department of Defense (1980s)
Syntax	C-like — concise	Ada-like — strict, verbose
Typing	Loose typing	Strong typing (strict checking)
Popular in	ASIC/FPGA industry (US, Asia)	Government/aerospace work (Europe)
Standard	IEEE 1364	IEEE 1076

Both languages can interoperate in the same project (mixed-language design) using modern EDA tools

2 · INTRODUCTION

What is Verilog?

Hardware Description Language (HDL)

- Verilog is a language for describing the structure and behavior of digital circuits, replacing hand-drawn schematics
- Used to design circuits from gate-level up to large systems (CPUs, SoCs)
- Compiled/synthesized into real circuits on FPGA or ASIC
- Supports both simulation and synthesis

Software vs Hardware Description

- Software: instructions run one line at a time (sequential execution)
- Hardware/HDL: every line operates 'concurrently' because it represents real circuitry
- Variables in HDL are real wires/registers, not memory locations
- 'Compiling' means synthesizing into a circuit, not building an executable

2 · INTRODUCTION

Why Use an HDL?

Why the industry chooses Verilog/VHDL over drawing circuits by hand

Circuit Scale

Modern circuits have millions of transistors — hand-drawn schematics simply can't keep up

Reusability

Write a module once, reuse it across many projects

Verification

Simulate to verify correctness before fabrication

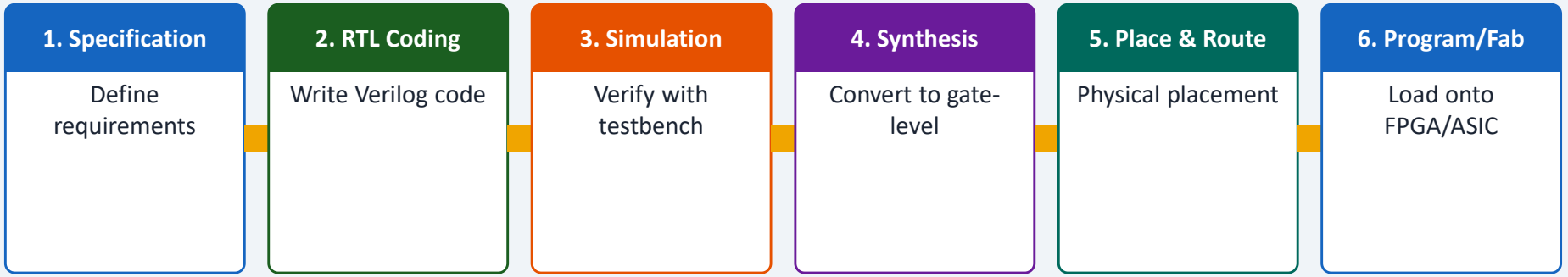
Portability

The same code synthesizes onto FPGA or ASIC across different technologies

2 · INTRODUCTION

Digital System Design Flow

Steps from initial concept to a real chip



Chapter 3 focuses mainly on step 2 (RTL Coding) — the essential foundation before next week's hands-on lab

Levels of Abstraction in Design

Verilog can be written at several levels, from gate to behavior

Behavioral	Describes circuit 'behavior' using always/if/case without specifying gates	`if (sel) y = a; else y = b;`
Dataflow	Describes how data flows using continuous assignment	`assign y = sel ? a : b;`
Structural	Directly connects gates or sub-modules together	`and g1(y, a, sel); or g2(...);`

2 · INTRODUCTION

RTL vs Testbench

Distinguishing synthesizable code from simulation-only code

RTL (Register-Transfer Level)

Code that describes real hardware. Must be synthesizable — translatable into actual gates, flip-flops, and wires by a synthesis tool. No delays (#) are used inside RTL.

```
// RTL — synthesizable
module and_gate(input a, b, output y);
  assign y = a & b;
endmodule
```

Testbench

Code that exists only to test the design in simulation. Never synthesized into hardware. Freely uses delays (#), $display, and other simulation-only constructs.

```
// Testbench — simulation only
module tb;
  reg a, b; wire y;
  and_gate dut(a, b, y);
  initial begin
    a=0;b=0;#10; a=1;b=1;#10;
    $display("y=%b", y);
  end
endmodule
```

2 · INTRODUCTION

MUX 2:1 in Three Modeling Styles

The same circuit, written three different ways

```
// 1. Behavioral
module mux_behav(
  input a, b, sel,
  output reg y
);
  always @(*) begin
    if (sel) y = b;
    else     y = a;
  end
endmodule
```

```
// 2. Dataflow
module mux_dataflow(
  input a, b, sel,
  output y
);
  assign y = sel ? b : a;

endmodule
```

```
// 3. Structural
module mux_struct(
  input a, b, sel,
  output y
);
  wire nsel, t1, t2;
  not g1(nsel, sel);
  and g2(t1, a, nsel);
  and g3(t2, b, sel);
  or  g4(y, t1, t2);
endmodule
```

All three implementations produce identical hardware after synthesis — the choice of style is about clarity and design intent, not function

3 · SPECIFICATION

Case Study: Designing an Arbiter Circuit

Learning the HDL design flow through a real 4-step example

a. Block Diagram	b. Low Level Design	c. Modules	d. Code
View overall input/output	Break down the operation	Split circuit into sub-blocks	Write actual Verilog code

Arbiter: a circuit that decides who gets to use a shared resource (e.g. a bus) when multiple requesters compete simultaneously

3 · SPECIFICATION

What is a Specification Document?

The document defining circuit requirements before any code is written

- Functional Spec — what the circuit must do (e.g. the arbiter must grant exactly one requester)
- Interface Spec — ports, signal direction, bit-width of each signal
- Timing Spec — clock speed, acceptable latency, setup/hold time
- Constraints — chip area budget, power consumption

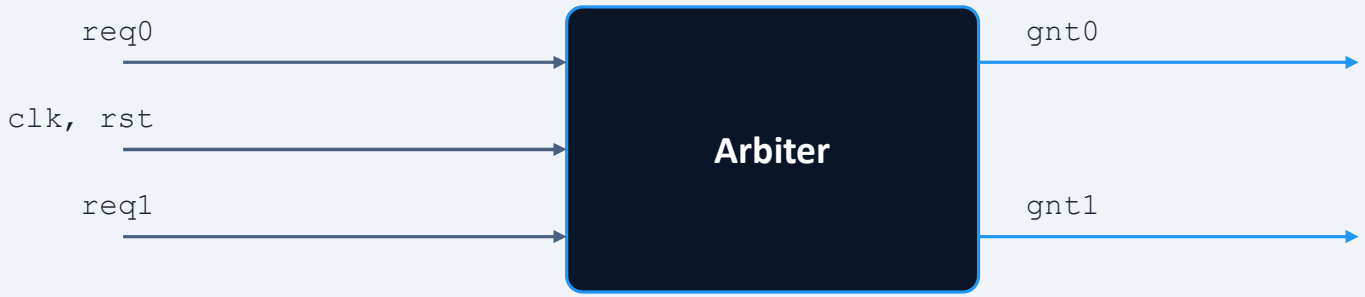
Why Spec Comes Before Code

- Prevents misinterpretation between the design team and verification team
- Becomes the basis for testbench checks later on
- Reduces costly rework (re-spins) — especially expensive in ASIC projects
- Serves as the starting point for the block diagram and low-level design steps

3A · BLOCK DIAGRAM

Block Diagram of Arbiter

Viewing the circuit from outside: what goes in, what comes out



- Input: req0, req1 (request signals from 2 devices), clk, rst
- Output: gnt0, gnt1 (grant signals — only one can be active at a time)
- A block diagram only describes the 'interface' — it doesn't yet describe internal behavior

3B · LOW LEVEL DESIGN

Low Level Design

Breaking down the internal operation of the Arbiter

- Define the decision rule: if req0 and req1 request at the same time, who wins?
- Use a simple priority scheme: req0 has higher priority than req1
- Design the state/logic: gnt0 = req0, gnt1 = req1 & ~req0
- Consider the edge case: no requests at all → nobody is granted

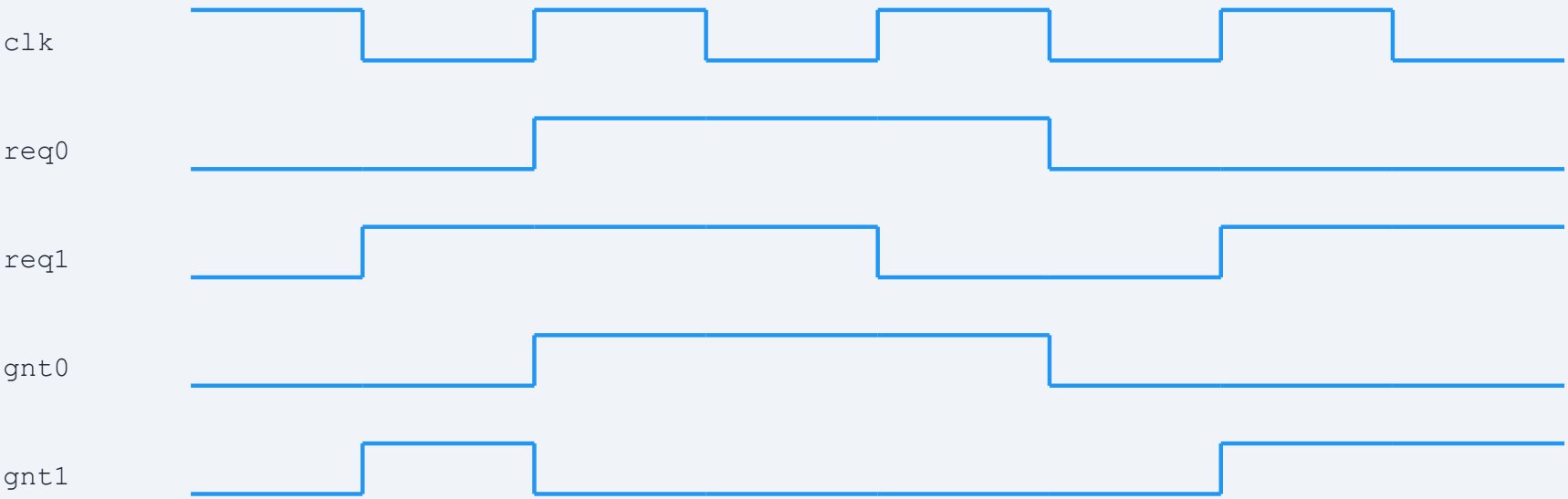
Truth Table (Priority Logic)

req0	req1	gnt0	gnt1
0	0	0	0
0	1	0	1
1	0	1	0
1	1	1	0

3B · LOW LEVEL DESIGN

Timing Behavior of the Arbiter

Waveform diagram showing how signals relate over time



- At clock cycle 3: req0=1, req1=1 → priority logic selects gnt0=1 (req0 wins)
- At clock cycles 7-8: req0=0, req1=1 → gnt1=1 per the condition req1 & ~req0

4 · MODULE

Modules: The Building Block of Verilog

Every circuit in Verilog is wrapped inside a module

- A module is a container that bundles ports, variable declarations, and logic together
- Each module has a clear interface: input / output / inout
- A module can instantiate other modules inside it (hierarchical design)
- Splitting into sub-modules makes testing and reuse much easier

```
module arbiter (
  input  wire clk, rst,
  input  wire req0, req1,
  output wire gnt0, gnt1
);
  // internal logic here
endmodule
```

4 · MODULE

Module Instantiation

Calling one module from another (hierarchical design)

```
module top (
  input  clk, rst, req0, req1,
  output gnt0, gnt1
);

  // Named port mapping (recommended)
  arbiter u_arb (
    .clk  (clk),
    .rst  (rst),
    .req0 (req0),
    .req1 (req1),
    .gnt0 (gnt0),
    .gnt1 (gnt1)
  );
endmodule
```

- Named port mapping (.port(signal)) is safer — order doesn't matter
- Positional mapping (ports listed in order) is shorter but more error-prone
- u_arb is the instance name — the same module can be reused multiple times
- Hierarchical design makes it easier to debug and test layer by layer

4 · MODULE

Code of Module "arbiter"

The complete Verilog code, from spec to implementation

```
module arbiter (
  input  wire clk,
  input  wire rst,
  input  wire req0,
  input  wire req1,
  output reg  gnt0,
  output reg  gnt1
);

  always @(posedge clk or posedge rst) begin
    if (rst) begin
      gnt0 <= 1'b0;
      gnt1 <= 1'b0;
    end else begin
      gnt0 <= req0;
      gnt1 <= req1 & ~req0;
    end
  end
endmodule
```

- Lines 1-8: module declaration and port list
- Line 10: always block triggers on every posedge of clk or rst
- Lines 11-14: asynchronous reset
- Lines 15-16: priority logic designed in the low-level design step

5 · DATA TYPES

Data Types in Verilog

Overview of the core data types: width, signed/unsigned, usage, synthesizable

Data Type	Width	Signed/Unsigned	Primary Use	Synthesizable
wire	User-defined (default 1-bit)	Unsigned (4-state: 0,1,X,Z)	Connecting hardware elements (nets)	Yes — represents physical wires
reg	User-defined (default 1-bit)	Unsigned (4-state: 0,1,X,Z)	Procedural assignments, storage	Yes — may infer flip-flops or combinational logic
integer	32 bits	Signed	Loop counters, arithmetic operations	Partial — primarily for testbenches
time	64 bits	Unsigned	Storing simulation time	No — simulation only
real	64 bits (IEEE 754 double)	Signed floating point	Floating point calculations, delays	No — simulation only
realtime	64 bits (IEEE 754 double)	Signed floating point	High-precision time measurements	No — simulation only

5 · DATA TYPES

wire and reg: The Two Most Common Types

Both are synthesizable, but behave fundamentally differently

wire

Represents a real physical wire, used with continuous assignment (assign). Stores no value — must always be driven (4-state: 0,1,X,Z)

reg

Can hold a value, used inside always/initial blocks — does not necessarily mean it becomes a hardware register (4-state: 0,1,X,Z)

```
wire        a, b;
wire [7:0]  data_bus;
reg         flag;
reg  [3:0]  counter;
parameter   WIDTH = 8;
localparam  MAX   = 15;
integer     i;
```

5 · DATA TYPES (CONT.)

wire vs reg Example

Why this is the #1 misconception for new Verilog students

```
// WRONG: wire cannot be assigned in always
module bad_example(input a, b, output wire y);
  always @(*)
    y = a & b;   // ERROR: y is a wire
endmodule

// CORRECT: use reg with always
module good_example(input a, b, output reg y);
  always @(*)
    y = a & b;   // OK: y is a reg
endmodule

// CORRECT: use wire with assign
module also_good(input a, b, output wire y);
  assign y = a & b;   // OK: y is a wire
endmodule
```

Key Misconception

- 'reg' does NOT always mean a real hardware register
- It just means 'a variable assigned inside a procedural block' (always/initial)
- Whether reg becomes a flip-flop or stays combinational depends entirely on the always block's sensitivity list
- Rule of thumb: assign → wire. always → reg.

5 · DATA TYPES

Scalar and Vector

The difference between a 1-bit signal and a multi-bit signal

Scalar

A 1-bit signal — no range specified at declaration (default width = 1)

Vector

A multi-bit signal — the range [MSB:LSB] is explicitly specified at declaration

- [MSB:LSB] defines the width, e.g. [7:0] is 8 bits, with the leftmost bit (7) as MSB
- The range direction can be [7:0] or [0:7], but must stay consistent throughout the file
- If no range is specified, the default is always a scalar (1 bit)

```
// Scalar (1 bit)
wire clk;
reg  enable;

// Vector (multiple bits)
wire [7:0]  data_bus;
reg  [31:0] instr;
wire [0:7]  little_end; // [0:7] also works
```

5 · DATA TYPES

Bit-Select and Part-Select

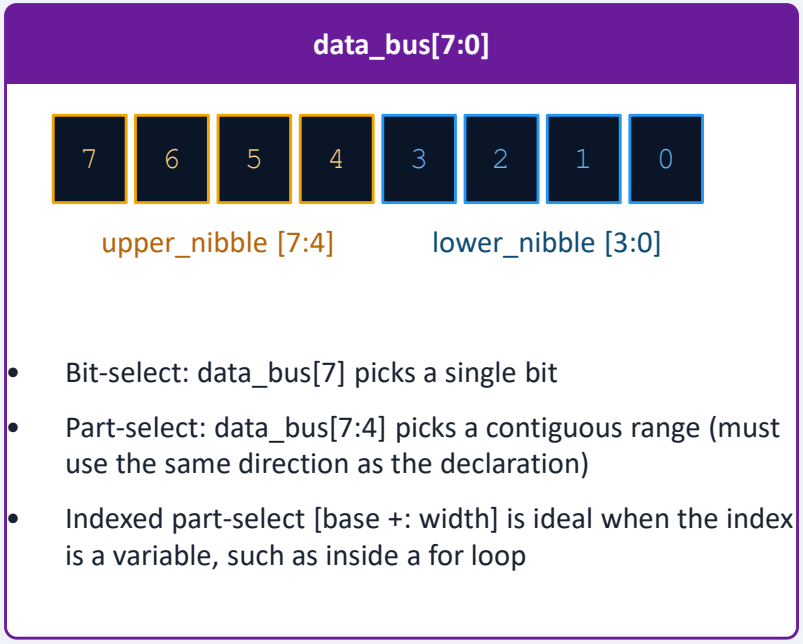
Accessing a single bit or a sub-range of bits from a vector

```
wire [7:0] data_bus;

// Bit-select: pick 1 bit
wire msb = data_bus[7];
wire lsb = data_bus[0];

// Part-select: pick a contiguous range
wire [3:0] upper_nibble = data_bus[7:4];
wire [3:0] lower_nibble = data_bus[3:0];

// Indexed part-select (variable index, fixed width)
wire [3:0] byte_part = data_bus[idx +: 4];
```



- Bit-select: data_bus[7] picks a single bit
- Part-select: data_bus[7:4] picks a contiguous range (must use the same direction as the declaration)
- Indexed part-select [base +: width] is ideal when the index is a variable, such as inside a for loop

5 · DATA TYPES

Number Literal Formats

How to write numeric constants in Verilog: <size>'<base><value>

Base	Symbol	Example	Meaning
Binary	'b	4'b1010	Base 2, 4 bits = 10
Hex	'h	8'hFF	Base 16, 8 bits = 255
Decimal	'd	8'd25	Base 10, 8 bits
Octal	'o	6'o17	Base 8, 6 bits

- If no size is specified, the system default of 32 bits is used (best avoided — always specify a size)
- x represents don't-care/unknown and z represents high-impedance; usable in simulation, e.g. 4'b10xz
- An underscore _ separates digits for readability and has no effect on the value, e.g. 16'b1010_1100_0011_1111

5 · DATA TYPES (CONT.)

Signed vs Unsigned Numbers

How Verilog represents negative numbers using two's complement

```
// Unsigned (default)
wire [3:0] u = 4'b1100;  // = 12

// Signed — must declare explicitly
wire signed [3:0] s = 4'b1100;  // = -4
reg  signed [7:0] temp;

// Arithmetic with signed values
wire signed [7:0] a = -8'sd5;
wire signed [7:0] b = 8'sd3;
wire signed [8:0] sum = a + b;  // = -2
```

Two's Complement Basics

- All values are unsigned by default in Verilog
- The 'signed' keyword tells the tool to treat the MSB as a sign bit
- Negative numbers are stored as two's complement: invert all bits, then add 1
- Mixing signed and unsigned operands in one expression silently converts everything to unsigned — a common source of bugs

6 · OPERATORS

Operators in Verilog

The core groups of operators used to build logic

Arithmetic	Logical	Relational	Bitwise
+ - * / %	&& || !	> < >= <= == !=	& | ^ ~ ^~

- Relational/Logical operators return 1 bit (true/false), used in conditions
- Bitwise operators act on each bit of the operand — the most common in digital circuits
- Arithmetic on vectors requires care around overflow and result bit-width

6 · OPERATORS (CONT.)

Logical vs Bitwise Operators

A very common source of bugs — these are NOT the same operators

Operator	Type	Operates on	Result width
&	Bitwise AND	Each bit of the operand	Same as operand width
&&	Logical AND	Whole value (treated as true/false)	Always 1 bit
|	Bitwise OR	Each bit of the operand	Same as operand width
||	Logical OR	Whole value (treated as true/false)	Always 1 bit

```
wire [3:0] a = 4'b1100, b = 4'b1010;
wire x = a & b;   // bitwise:  x = 4'b1000
wire y = a && b;  // logical:  y = 1'b1 (both nonzero = true)
```

6 · OPERATORS

Reduction, Shift, Concatenation, Conditional

Special operators commonly used in hardware design

```
// Reduction (collapse all bits to 1 bit)
y = &data;     // AND of all bits
y = |data;     // OR of all bits

// Shift
y = data << 2; // shift left 2 bits
y = data >> 1; // shift right 1 bit

// Concatenation & conditional
y = {a, b, c};       // bits joined together
y = (sel) ? a : b;   // conditional (mux)
```

- Reduction operators are often used to check whether all bits are 1 or whether any bit is 1
- Shift operators implement simple multiplication/division by powers of 2
- Concatenation { } combines several signals into a single bus
- Conditional (?:) synthesizes directly into a multiplexer

6 · OPERATORS (CONT.)

Concatenation Example

Building a wider bus out of smaller pieces

```
wire [3:0] high_byte = 4'hA;
wire [3:0] low_byte  = 4'h5;
wire [7:0] full_byte;

assign full_byte = {high_byte, low_byte};
// full_byte = 8'hA5

// Replication operator: repeat a pattern N times
wire [3:0] all_ones = {4{1'b1}};  // = 4'b1111
wire [7:0] zero_pad = {4'b0000, low_byte};

// Common use: sign extension
wire signed [3:0] small = -4'sd3;
wire signed [7:0] extended = {{4{small[3]}}, small};
```

Reading the Diagram

- {a, b} joins signals left to right: the first item becomes the most significant bits
- {N{pattern}} repeats pattern N times — handy for sign extension or zero padding
- The result width equals the sum of all the concatenated widths
- Concatenation can appear on either side of an assignment

6 · OPERATORS

Operator Precedence

Order of evaluation when multiple operators appear in one expression (high → low)

Rank	Operator	Category
1 (highest)	! ~	Logical/Bitwise NOT
2	* / %	Arithmetic
3	+ -	Arithmetic
4	<< >>	Shift
5	< <= > >=	Relational
6	== !=	Equality
7	&	Bitwise AND
8	^ ^~	Bitwise XOR/XNOR
9	|	Bitwise OR
10	&&	Logical AND
11	||	Logical OR
12 (lowest)	?:	Conditional

6 · OPERATORS

Example: Combining Operators in a 2:1 Mux

Using bitwise + conditional operators to build a real circuit

```
module mux2to1 (
  input  a, b, sel,
  output y
);
  // Method 1: conditional operator
  assign y = sel ? b : a;

  // Method 2: bitwise (equivalent)
  // assign y = (sel & b) | (~sel & a);
endmodule
```

sel	a	b	y
0	0	x	0
0	1	x	1
1	x	0	0
1	x	1	1

x = don't-care — the result depends only on sel in each case

Session 1 — Summary

History — Verilog's origins, the path to IEEE standardization, Verilog vs VHDL ✓

Introduction — what HDL is, why it's used, RTL vs testbench, abstraction levels, MUX in 3 styles ✓

Specification — the Arbiter case study: spec document, block diagram, low-level design, timing ✓

Module — the building block of Verilog, instantiation, hierarchical design ✓

Data Types — wire/reg, scalar/vector, bit-select, signed/unsigned, number literals ✓

Operators — arithmetic, logical, bitwise, concatenation, precedence ✓

Coming next: Session 2 — Combinational Design (Assign, If-Else, Case, Loop Statements)